CO 1 – ASSESSMENT TOOL -1

CODING CHALLENGE

COURSE CODE: **DSA0308**

COURSE NAME: **NLP**

Reg no**:192425108** Name: **K.Sadiq hussain**

---

1) Intelligent Email and Password Validator Using Regular Expressions

Program:

```
import re
def validate_email(email):
    pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$'
    return re.fullmatch(pattern, password) is not None
def validate_mobile(mobile):
    pattern = r'^[6-9]\d{9}$'
    return re.fullmatch(pattern, mobile) is not None
def main():
    print("EMAIL, PASSWORD AND MOBILE VALIDATOR")

    email = input("Enter email: ")
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ")

    if validate_email(email):
        print("Valid Email")
```

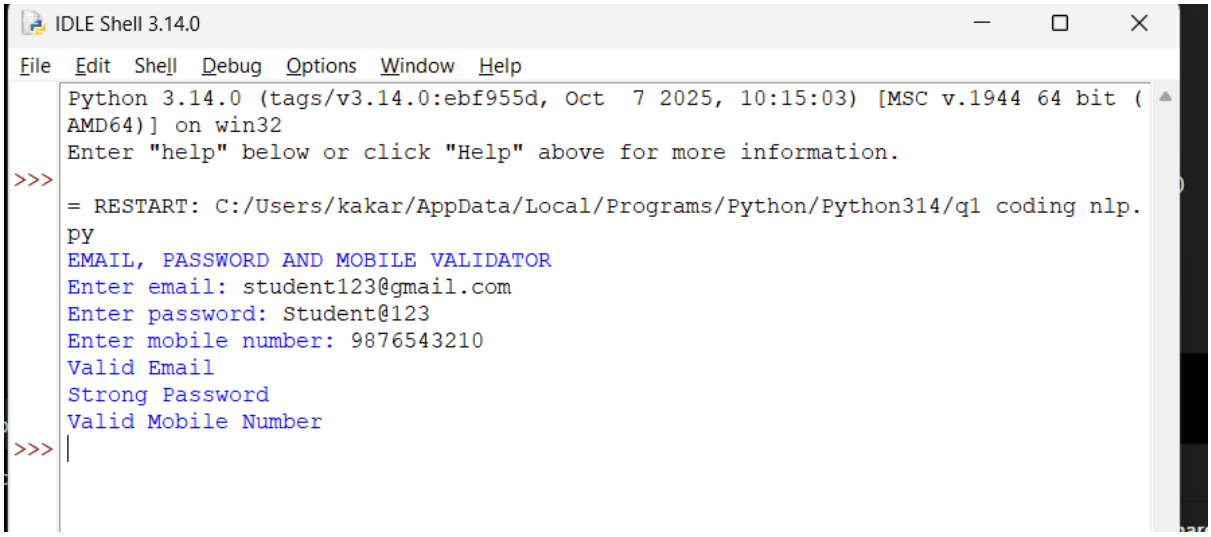
```
    else:
        print("Invalid Email")

    if validate_password(password):
        print("Strong Password")
    else:
        print("Weak Password")

    if validate_mobile(mobile):
        print("Valid Mobile Number")
    else:
        print("Invalid Mobile Number")


if __name__ == "__main__":
    main()
```

OUTPUT:



2) Design a Finite State Automata (FSA) Simulator

Program:

```
def simulate_dfa(string):
    transition = {
        'q0': {'a': 'q1', 'b': 'q0'},
        'q1': {'a': 'q1', 'b': 'q2'},
        'q2': {'a': 'q1', 'b': 'q0'}
    }

    current_state = 'q0'
    final_states = ['q2']
    path = [current_state]

    for symbol in string:
        if symbol not in ['a', 'b']:
            print("Invalid input symbol:", symbol)
            return

        current_state = transition[current_state][symbol]
        path.append(current_state)

    print("Transition Path:", " -> ".join(path))

    if current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")


def main():
```

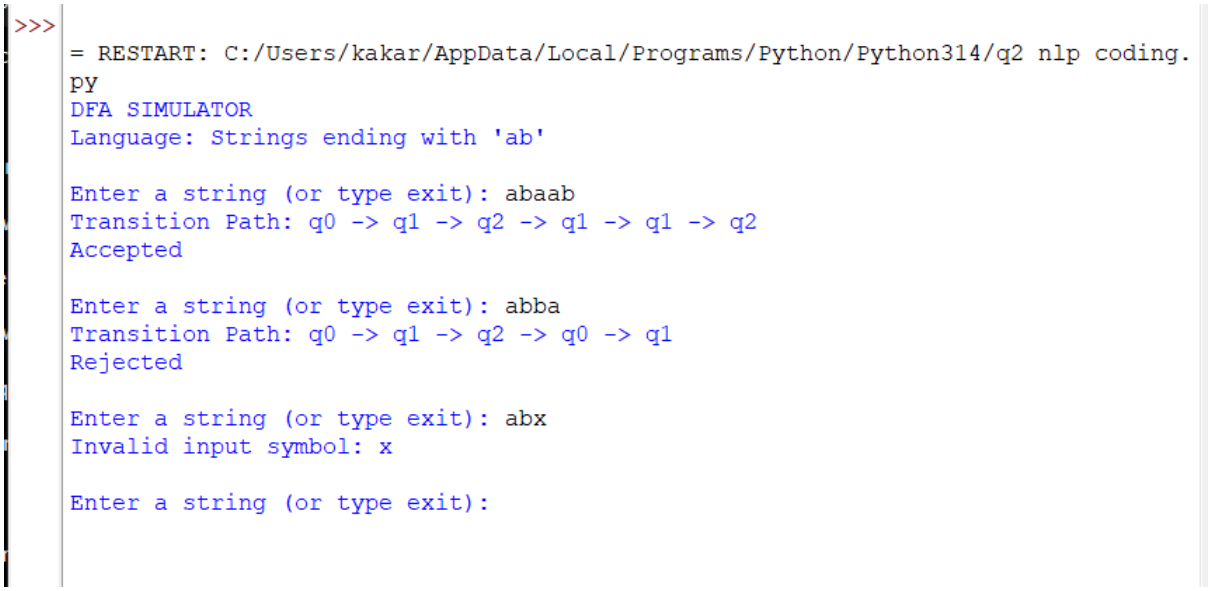
```
    print("DFA SIMULATOR")
    print("Language: Strings ending with 'ab'")

    while True:
        string = input("\nEnter a string (or type exit): ")

        if string.lower() == "exit":
            break

        simulate_dfa(string)
if __name__ == "__main__":
    main()
```

OUTPUT :



3) Smart Pattern Matching Engine Using Regular

Program:

```
import re
def search_pattern(text, choice):
    if choice == '1':
        pattern = r'\b\d{2}/\d{2}/\d{4}\b'

    elif choice == '2':
        pattern = r'\b[6-9]\d{9}\b'

    elif choice == '3':
        pattern = r'(?<!\w)#[A-Za-z0-9_]+'

    elif choice == '4':
        pattern = r'(?<!\w)@[A-Za-z0-9_]+'

    elif choice == '5':
        prefix = input("Enter prefix: ")
        pattern = r'\b' + re.escape(prefix) + r'\w*'

    elif choice == '6':
        suffix = input("Enter suffix: ")
        pattern = r'\b\w*' + re.escape(suffix) + r'\b'

    else:
        print("Invalid choice")
        return

    matches = re.findall(pattern, text, re.IGNORECASE)
```

```
    if matches:
        print("Matching patterns:", matches)
    else:
        print("No matching patterns found")
def main():
    print("SMART PATTERN MATCHING ENGINE")

    text = input("Enter text: ")

    while True:
        print("\n1. Search Date")
        print("2. Search Phone Number")
        print("3. Search Hashtag")
        print("4. Search Mention")
        print("5. Search Prefix")
        print("6. Search Suffix")
        print("0. Exit")

        choice = input("Enter your choice: ")

        if choice == '0':
            print("Exiting program")
            break

        search_pattern(text, choice)


if __name__ == "__main__":
    main()
```

OUTPUT:

```
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/q3 nlp coding.
py
SMART PATTERN MATCHING ENGINE
Enter text: meeting on 12/09/2026

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
0. Exit
Enter your choice: 1
Matching patterns: ['12/09/2026']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
0. Exit
```